

عمل الطالب: بشار حافظ علوان

SOFTWARE ENGINEERING

اشراف المهندس

مالك المصنف

ENGINEERING



نبذة تعريفية

هناك عدّة محركات قوالب بديلة للغة القوالب الافتراضية في جانغو، من أبرزها: جينجا، ماکو، کاميليون، وتشيتا. تختلف هذه المحركات من حيث السرعة، وأسلوب الكتابة، ومدى توافقها مع إطار جانغو بعضها يركّز على الأداء والمرونة مثل جينجا وماكو، وبعضها يهتم بالتوافق مع المعايير الرسمية مثل کاميليون. اختيار المحرّك الأنسب يعتمد على طبيعة المشروع، ومدى حاجته إلى الأداء أو التوافق أو سهولة الصيانة.

01 JINJA2

محرك قوالب مرّن وسريع جدًّا، مكتوب بلغة بايثون، ويستخدم بشكل افتراضي في Django ويمكن دمجه مع Flask، في بسهولة.



04 CHEETAH

واحد من أقدم محركات القوالب في بايثون. يدعم تضمين بايثون داخل ويستخدم في تطبيقات معينة HTML قديمة.

03 CHAMELEON

يُعرف بسرّعه وامتناله للمعايير، ويستخدم غالبًا في المشاريع الرسمية أو المؤسسية. يوفر أدوات تساعد على تنظيم قالب بطريقة دقيقة ومنضبطة. يُناسب الأنظمة التي تتطلب توافقًا مع معايير صارمة أو بيئات قديمة.

02 MAKO

محرك قوالب مكتوب بلغة بايثون، يتميز بأداء ممتاز، ويتيح تضمين كود بايثون مباشرة داخل قالب.

مزايا القوالب

JINJA2

المزايا

- أداء عالي جدًا
- يقوم بتحويل القوالب إلى بايثون
- تركيب قريب من بايثون: ما يجعله سهلًا للمطورين
- macros، inheritance، filters، includes،
- يسمح بتضمين منطق معقد نسبيًا في القوالب

MAKO

المزايا

- أداء ممتاز
- يسمح بكتابة كود بايثون صريح داخل قالب (قوة ومرونة)
- دعم ممتاز للقوالب الديناميكية الكبيرة

CHAMELEON

المزايا

- سريع جدًا: يولد bytecode يدعم منطقًا قويًا داخل القوالب (مثل if و loop و inheritance)
- يحترم قواعد HTML/XHTML بالكامل (مناسب للبيئات الرسمية)

CHEETAH

المزايا

- مرن جدًا من حيث تضمين منطق بايثون
- قابل للتوسعة بدرجة عالية
- يستخدم بشكل مشابه لما كان قديمًا ASP و PHP يُستخدم في





العيوب

Jinja2



يحتاج إلى إعداد مخصص في Django
تضمنين منطق كثير في القوالب قد
يُعد الصيانة

Chameleon

أقل شيعة من DTL
قد يكون أكثر تعقيدًا من ناحية التركيب
والتخصيص
قد لا يتكامل مع أدوات Django

Mako

كتابة بايثون داخل القالب يمكن أن يؤدي
إلى قوالب غير قابلة للصيانة
ليس مهياً للعمل مع Django بشكل
افتراضي، يتطلب تكامل يدوي

Cheetah

- صعب الدمج مع Django
- قد لا يكون آمنًا بشكل كافٍ دون تعديل
- المجتمع والدعم محدود جدًا



الاستخدام

Jinja2

- مشاريع تعتمد على قوالب ديناميكية جدًا.
- تطبيقات تحتاج أداءًا عاليًا، أو تتطلب واجهات تفاعلية معقدة.

١

Main Office

- المشاريع التي تحتاج تحكمًا دقيقًا في الأداء.
- تطبيقات داخلية أو أدوات حيث الصيانة ليست أولوية قصوى.
- أنظمة تعتمد على قوالب مركبة ومعقدة جدًا.

Chameleon

- المؤسسات التي تعتمد على معايير صارمة (XML، XHTML).
- المشاريع التي تحتاج أداء عالٍ ومعايير صارمة للعرض.
- أنظمة مؤسسية أو حكومية قديمة تحتاج توافقًا خاصًا.

Cheetah

- الأنظمة القديمة التي بنيت عليه.
- إذا كنت تهاجر مشروعًا قديمًا يعتمد على Cheetah إلى Django.

جدول المقارنة

المعيار	Jinja2	Mako	Chameleon	Cheetah
اللغة	Pythonic + شبيهة بـ Django Template	Pythonic تمامًا	XML-based (شبيهة بـ ZPT)	Pythonic
الأداء	عالي	أعلى من Jinja2 في بعض الحالات	ممتاز، خاصة مع التخزين المؤقت	جيد، لكن ليس الأفضل
سهولة الاستخدام	سهولة وقريبة من Django Template	متقدمة قليلاً، تتطلب خبرة أكبر	بسيطة جدًا لكن محدودة في التعابير	بسيطة لكن غير شائعة
التكامل مع Django	مدعومة رسميًا (من Django 1.8)	غير مدعومة رسميًا	غير مدعومة رسميًا	غير مدعومة رسميًا
القابلية للتوسيع	عالية (فلاتر، توسيع بسهولة)	عالية (كود Python مباشر)	محدودة نسبيًا	قابلة لكن ليست مريحة
الأمان (تلقائي)	نعم (Escaping افتراضي)	لا (يجب الحرص يدويًا)	نعم (مثل Jinja2)	لا
التوثيق والمجتمع	كبير وممتاز	جيد، لكن أصغر من Jinja2	محدود	ضعيف جدًا
التوافق مع HTML	ممتاز	ممتاز	ممتاز	جيد
نموذج البرمجة	فصل جيد بين المنطق والعرض	يسمح بمنطق أكثر داخل القالب	موجه لعرض فقط	يشبه Mako لكنه أقدم
الاستخدام في المشاريع	Flask, Django, Ansible, وغيرها	Pyramid, وبعض الأنظمة المتقدمة	Plone, بعض الأنظمة السريعة	نادر جدًا



THE END

THANK YOU

THANK YOU